

Purchasing Agent

Approach and design write-up — AI Purchasing Agent assignment (Rappi Turbo)

Source code (public GitHub repository): <https://github.com/aashishky2/ai-purchasing-agent>

What this document is

This is a plain-language walkthrough of the assignment submission: what was asked, what was built, why it was built that way, and how to check it actually works. The repository linked above has the full source code, a shorter README, and the commit history. This document is meant to be readable on its own, without needing to open the code first.

The assignment, in one paragraph

A retail / quick-commerce buyer normally has to manually check inventory, demand forecasts, open orders, supplier terms, budget, and storage space before deciding what to purchase. The task was to build a full-stack agent that does this investigation itself, makes a purchasing decision, carries out that decision (for example, creating a purchase order), and then checks whether the outcome was actually correct — without assuming the system's original recommendation was right in the first place.

What was built, in one paragraph

A working agent that covers all four scenarios described in the assignment, built around one shared decision loop instead of four separate pieces of logic. It reads live data (inventory, demand history, suppliers, budget, storage) from a small seeded database, decides what to do using a transparent, rule-based decision engine, takes the action itself when it's confident and the stakes are low, and asks a human buyer for sign-off when the situation is unusual, expensive, or has no safe automatic answer. After acting, it double-checks its own work — and in one of the five example situations, it actually catches a real problem with its first attempt and fixes itself. There's a browser page to click through all of this, and a small test suite that checks the agent's behaviour automatically.

How the agent thinks: the five-step loop

Every situation the agent handles goes through the same five steps. This is the core idea of the whole project, so it's worth explaining simply, with an everyday comparison: it's the same process a careful human buyer would use, just done consistently and quickly every time.

1. Perceive Look up the facts: how much stock is there, how fast is it selling, what's already on order, what do suppliers charge and how long do they take, how much money and shelf space is left.	2. Reason Work out what should happen, using a fixed set of rules — not a guess. The same facts always lead to the same decision.
3. Act If the decision is safe and small enough, do it immediately: create or adjust a purchase order. If not, don't guess — stop and ask a human.	4. Validate After acting, check again: did the thing you assumed (like available storage space) actually still hold true? If something changed, fix it or escalate — never leave a silently broken order behind.

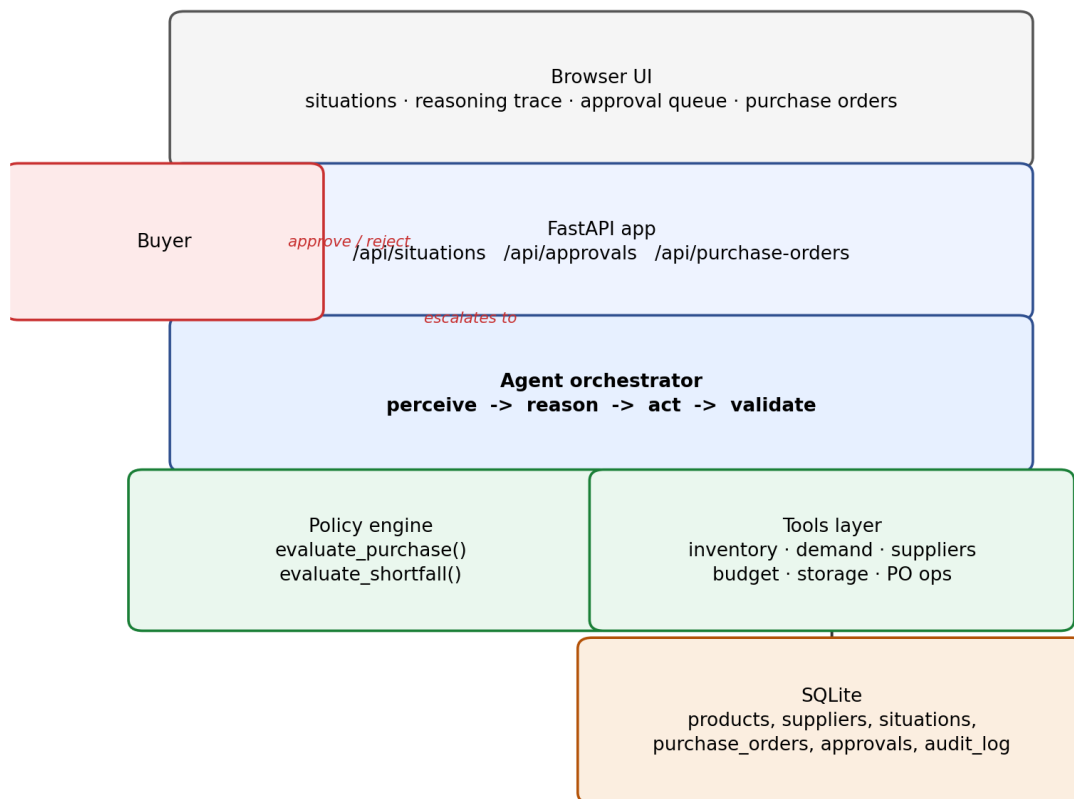
A fifth step, **escalate**, is really a fork in the road inside steps 3 and 4: any time the agent isn't confident, or the situation is expensive, or something goes wrong after acting, the buyer sees exactly what the agent saw and why it's unsure, and makes the final call from the browser page.

How the four scenarios map onto this loop

All four scenarios from the assignment use the same loop above. Scenarios 1, 3, and 4 all really ask the same underlying question — "how much should we buy, from whom, right now?" — just triggered by different starting facts, so they share one decision function (`evaluate_purchase`) instead of three copies of similar logic. Scenario 2 is a different kind of event (a supplier replying with less than what was ordered) so it has its own decision function (`evaluate_shortfall`), but flows through the same act/validate/escalate steps.

Scenario	What starts it	Which logic decides
1 — Recommendation review	The system suggests buying a certain quantity	<code>evaluate_purchase</code>
2 — Supplier can't fulfil	A supplier confirms less than what was ordered	<code>evaluate_shortfall</code>
3 — Demand has changed	No standing suggestion yet; actual sales have moved	<code>evaluate_purchase</code> (using real sales, not the old forecast)
4 — Purchasing constraint	A suggestion exists but budget/storage won't allow it	<code>evaluate_purchase</code> (constraint blocks it)

Architecture



Backend

Python, using FastAPI for the web server and plain SQLite for storage (no separate database server needed — it's a single file). Every time the server starts, the database is wiped and reloaded from one seed file (`seed_data.json`), so the demo always starts from the same known state and is easy to re-run.

Frontend

One self-contained web page (plain JavaScript, no build tools required — just open it in a browser once the server is running). It shows the list of situations, a button to run the agent, the full step-by-step reasoning behind its decision, the resulting purchase order, and a queue of anything waiting on a human.

Why a rule-based decision engine instead of calling a language model at decision time

This is probably the single biggest design choice in the project, so it deserves a direct explanation. The decision engine (the part that decides accept / modify / reject / escalate) is written as ordinary, readable code with fixed rules and thresholds — not a live call to an AI model. Three reasons:

- **Same input, same output, every time.** A system that spends real money and commits real warehouse space needs to behave predictably. A rule-based engine can be re-run a thousand times on the same situation and always produce the same, explainable answer.
- **It can be tested like normal code.** The whole test suite runs in under a second, with no API key, no internet connection, and no risk of a flaky or inconsistent answer changing the test result from one run to the next.
- **It's easier to audit.** Every decision returns the exact list of facts and thresholds that produced it (see the worked examples below) — there's nothing hidden inside a model's reasoning that a buyer or an engineer would have to trust blindly.

This doesn't mean a language model has no place here — it's just not needed for the actual accept/modify/reject decision. The natural places to add one, without touching the decision rules at all, are: turning a supplier's free-text email into the structured "only 250 of 500 available" event this agent already knows how to handle, or rephrasing the agent's decision into a friendlier paragraph for a specific buyer. The code is structured so either could be added later behind the existing tool boundary.

Walking through all four scenarios with real numbers

Below are the actual example situations loaded into the demo, with the real numbers the agent works with. These are also exactly what running the app or the test suite will show — nothing here is simplified from what actually happens in the code.

Scenario 1 — Recommendation review (product: Insulated Water Bottle)

The purchasing system suggests ordering **800 units**. The agent looks up the facts: 150 units on the shelf, 100 more already on their way, and typical sales of about 40 units a day. Using the fastest reliable supplier (7-day delivery), it works out that only about **338 units** are actually needed to stay safely stocked until the next order could arrive — well under the suggested 800. On top of that, the warehouse only has **300 units** of free space left, which is an even tighter limit than the 338 figure. So the agent doesn't accept 800 and doesn't accept 338 either — it orders **300**, the largest amount that actually fits, and explains both reasons (over-ordering versus the real need, and the storage limit) in its output. Because 300 units is a small, low-risk order, it places the purchase order itself, immediately, without waiting for a human.

This situation also demonstrates the validation step: right after placing the order, the agent double-checks the storage figure and finds that 40 of those 300 units of space were taken by something else in the meantime, leaving only 260. Since 260 is still enough to meet the supplier's minimum order size, the agent quietly corrects its own purchase order from 300 down to 260 and records why — see the dedicated section on the feedback loop below for the full story.

Scenario 1, second example — a recommendation that's already correct (Reusable Shopping Bag)

The system suggests **150 units**. This time, the agent's own calculation lands on almost the same number (about 144), and nothing about budget or storage gets in the way. Rather than nitpicking a small difference, the agent accepts the recommendation as-is and places the order. This pairs with the water bottle example above to show the agent isn't simply "always second-guessing" — it only changes a recommendation when the numbers actually say it should.

Scenario 2 — Supplier can't fulfil the order (product: Instant Noodles Family Pack)

An order for **500 units** was placed with a supplier (Golden Harvest Foods). The supplier comes back and says only **250** can actually be delivered right now — a shortfall of 250 units. The agent first checks whether that's actually a problem: with 80 units already on the shelf plus the 250 that are coming, that's 330 units on hand, enough for about 5.1 days of typical sales. But a brand new order, even from the fastest supplier, takes at least 5 days to arrive, which doesn't leave a safe buffer. So the shortfall does need to be covered. The agent then looks for another supplier who can fill just the 250-unit gap: Pacific Foodstuffs can, at \$3.20 per unit versus the original \$3.00 — a 6.7% price increase, costing an extra \$800 in total. Because that price increase is under the 10% ceiling the agent is allowed to accept on its own, and the extra cost is well under the \$5,000 auto-approval limit, it places the make-up order with the new supplier immediately, with no human needed.

If no other supplier had enough stock, or the price increase had been too steep, the agent would have stopped short of ordering and handed the decision to a buyer instead, along with its full analysis — this exact situation (no alternate supplier available) is one of the automated tests.

Scenario 3 — Demand has changed (product: AA Batteries, 4-pack)

Normal daily sales for this product are about 30 units a day, and that's what the forecast still assumes. But actual sales over the last 3 days came in at 76, 80 and 82 units a day — nearly three times normal. Before

trusting a spike like this, the agent requires it to hold for at least 3 days in a row, so a single unusual day doesn't trigger an overreaction (this exact rule is tested separately with a one-day blip that correctly does *not* trigger a big reorder). Because the elevated sales have now held for 3 straight days, the agent treats it as real and re-does its math using the new, higher rate. With 200 units on the shelf and 150 more already coming, that's only about 4.4 days of cover left at the new sales pace — but a fresh order takes 9 days to arrive. That's a real gap, so the agent calculates a new order of about **935 units** is needed. Even though the resulting order is inexpensive, the agent always sends a demand-driven recommendation like this to a human buyer for a quick sign-off, on the principle that overriding the official forecast deserves a second pair of eyes — cheap or not.

Scenario 4 — A purchasing constraint blocks the obvious action (product: Phone Charging Cable)

The system suggests ordering **1,000 units**. Working from actual stock (40 units) and typical sales, the agent calculates roughly 385 units are genuinely needed. But the remaining budget for this category this cycle is only **\$300**, and the cable costs \$5 each — enough for just 60 units. On top of that, this supplier won't sell fewer than 100 units at a time (their minimum order size). Sixty is below that minimum, so there is no order size that is both affordable and acceptable to the supplier. Rather than either placing an order it can't actually pay for, or silently doing nothing and letting the product run out, the agent stops and escalates: it lays out the numbers for a buyer and leaves the choice to them — ask for extra budget this cycle, delay the purchase, or find a different supplier with a smaller minimum order.

The feedback loop, in detail

The assignment specifically asks how the system checks whether its own decision actually worked, and what happens if it didn't — this is that answer, using the water bottle example (Scenario 1) step by step, since it's the one where the correction actually happens.

- 1 The agent decides to order 300 units (see the worked example above) and places the purchase order right away, since it's a small, low-risk order.
- 2 Immediately afterward — simulating the moment the order is actually confirmed — the agent checks the storage figure again. In real life, some other process (another delivery, another team's order) could easily have used up some of that space in the meantime; the seed data recreates exactly that by having 40 of the 300 units of space taken in between the two checks.
- 3 The second check now shows only 260 units of space, not 300. The agent notices its own order no longer fits.
- 4 Because 260 is still at or above the supplier's minimum order size (150), the agent doesn't need a human for this — it simply lowers its own purchase order from 300 to 260 units and writes down exactly why, so anyone looking at the order later can see it was adjusted and understand the reason.
- 5 If shrinking the order down to fit had gone below the supplier's minimum, the agent would not guess — it would mark the order as an exception and hand it to a human instead of leaving a broken or silently-wrong order behind.

The same pattern (check → if wrong, fix it if it's safe to, otherwise escalate) is really what Scenario 2 is, one step earlier in the process: a supplier shortfall is just a case where a purchase order already didn't turn out the way it was expected to. And it's exactly what stops Scenario 4 from ever placing an order the numbers don't actually support.

When does a human get involved?

The agent is allowed to act completely on its own only when all of the following are true: the resulting order costs less than \$5,000, recent sales look normal (no unconfirmed spike), and there's a quantity that satisfies both the budget/storage limits and the supplier's minimum order size. If any of that isn't true, the agent stops short of acting and instead shows a buyer its full reasoning and proposed action, and waits for an approve or reject. This includes: a demand spike that's real but hasn't lasted 3 days yet (still too early to trust); a demand spike that has lasted 3 days (trusted, but still shown to a human, since a change this size deserves a second opinion even when the order itself is cheap); a situation with no order size that is both affordable and acceptable to the supplier; and an alternate-supplier price increase, on a shortfall, of more than 10%. These specific numbers (\$5,000, 10%, 3 days) are simple settings at the top of one file in the code, not hard-wired throughout the system, so they're easy to change and easy to discuss.

How to run it

From inside the repository:

```
cd backend
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload
```

Then open `http://localhost:8000` in a browser. Every time the server starts, the database is reset and reloaded from the seed file, so it always begins from the same five example situations described above. Click any situation, click **Run agent**, and the page shows the full step-by-step reasoning, the decision, the resulting purchase order (if any), and the outcome of the validation check. Anything waiting on a human shows up in the approval queue on the same page with Approve and Reject buttons. No API keys or environment variables are needed to run any of this.

Automated tests and the evaluation scorecard

There are two ways the behaviour above is checked automatically, both inside `backend/tests/`:

test_scenarios.py runs each of the five example situations through the real agent (not a simplified stand-in) and checks the decision and outcome match what's expected — including checking the exact corrected quantity (260) in the storage self-correction example above, so that behaviour is locked in by a test, not just something that happens to work in a demo. Two extra cases are checked directly against the decision rules with made-up numbers, for situations not worth building a whole example around: a supplier shortfall with no alternate supplier available at all, and a demand spike that's only lasted one day (which correctly does *not* get treated as a real trend yet).

evaluate.py is a small, separate script that runs all five example situations and prints a plain scorecard, which is what the assignment asks for — not a complicated evaluation framework, just a readable table showing whether each situation's decision and outcome matched expectations:

Situation	Decision	Match	Status	Match	Action taken	Validated	Sent to human
SIT-1001	modify	yes	resolved (adjusted)	yes	yes	yes	no
SIT-1002	accept	yes	resolved	yes	yes	yes	no
SIT-2001	sourced alternate	yes	resolved	yes	yes	yes	no
SIT-3001	recommend purchase	yes	pending approval	yes	no	yes	yes
SIT-4001	escalate	yes	escalated	yes	no	yes	yes

For every situation, this checks six things: was the decision the expected one, did it reach the expected outcome status, did it take action (or correctly decide not to), did the after-the-fact check pass (or correctly fail and stop), and did it involve a human exactly when it was supposed to. Those six questions are directly the same ones the assignment's evaluation section asks.

What data the demo runs on

Everything lives in one readable file, `backend/seed_data.json`: 5 products, 7 suppliers (with their price, delivery time, minimum order size, reliability, and how much stock they have available), 3 orders already in progress, and the 5 example situations described earlier in this document. Nothing is hidden in a database that needs a special tool to inspect — it's all in that one file.

Guide to the repository

Path	What it is
<code>README.md</code>	Shorter version of this write-up, meant to be read alongside the code
<code>backend/app/policy.py</code>	The actual decision rules — where every threshold in this document lives
<code>backend/app/agent.py</code>	The perceive / reason / act / validate loop that ties everything together
<code>backend/app/tools.py</code>	Every lookup and action the agent is allowed to perform
<code>backend/app/main.py</code>	The web API the browser page talks to
<code>backend/seed_data.json</code>	All the example data described above, in one place
<code>backend/tests/test_scenarios.py</code>	Automated tests for every scenario
<code>backend/tests/evaluate.py</code>	The scorecard script shown above
<code>frontend/index.html</code>	The browser page — one file, no build step

Honest limitations, and what would change with more time

- This handles one buyer, one warehouse, with no login or access control — it's built to show the decision-and-validation loop clearly, not to be a multi-team production system.
- Supplier replies, budget, and storage numbers are all simulated from the seed file rather than connected to real systems. The tools file is exactly where a real inventory system, supplier API, or budgeting system would be plugged in instead, without changing the decision rules at all.
- The "storage changed between decision and confirmation" moment used to demonstrate the feedback loop is deliberately seeded into the example data so it's reproducible on demand, rather than two real processes racing each other. The situation it represents — two systems reading a shared number a moment apart — is a completely normal, real thing to plan for.
- With more time: connect a real forecasting feed for Scenario 3 instead of a fixed trailing average, add a second warehouse/node to show the agent choosing between locations, and add the optional language-model step (mentioned above) for turning free-text supplier emails into structured events automatically.